

UNIVERSITÀ
degli STUDI
di CATANIA

UNIVERSITÀ DEGLI STUDI DI CATANIA

Dipartimento di Matematica e Informatica

Corso di Laurea Triennale in Informatica

Andrea Alecci

OWASP TOP 10: A04-2021 INSECURE DESIGN

INDICE

1.DEFINIZIONE DI INSECURE DESIGN.....3
1.1 Differenza tra insecure design ed insecure implementation.....3
2. SECURITY PRINCIPLE.....4
2.1 least privileges.....4
2.2 Separation of duty.....4
2.3 Defense in depth.....4
2.4 Zero trust.....5
2.5 Security in the open.....5
3 FOCUS SULLE AREE DI SICUREZZA.....6
3.1 CONTEXT.....6
3.2 COMPONENTS.....6
3.3 CONNECTION.....6
3.4 CODE.....7
3.5 CONFIGURATION.....7
4 THREAT MODELLING.....8
4.1 Threat modelling manifesto.....8
4.2 Fasi del threat modelling.....9
4.2.1 Modellazione del sistema.....9
4.2.2 Individuazione delle minacce.....9
4.2.3 Risposta e mitigazioni.....10
5 ECOMMERCE – DEMO.....11
5.1 INTRODUZIONE ALLA DEMO.....11
5.2 IDOR.....11
5.2.1 Definizione IDOR.....11
5.2.2 IDOR Attacco.....12
5.2.3 Conseguenze.....16
5.2.4 Mitigazioni.....16
5.3 CWE-602 - Client-Side Enforcement of Server-Side Security DEMO.....17

5.3.1 Definizione CWE-602.....17
5.3.2 CWE-602 Attacco.....17
5.3.3 Conseguenze.....20
5.3.4 Mitigazioni.....21
6 CONCLUSIONI.....22
BIBLIOGRAFIA.....23

1.DEFINIZIONE DI INSECURE DESIGN

L'insecure design si riferisce ad un'ampia categoria di debolezze che nascono da difetti di progettazione e dalla mancanza di controlli di sicurezza. A04-Insecure Design fa parte della OWASP top 10 2021, un documento che rappresenta una classifica delle 10 minacce più critiche per la sicurezza delle applicazioni web.

1.1 Differenza tra insecure design ed insecure implementation

insecure design e insecure implementation potrebbero sembrare concetti simili, tuttavia indicano problemi differenti seppur collegati. L'insecure design rappresenta un errore nella logica di un progetto, quindi anche se l'implementazione è impeccabile, si basa comunque su una logica errata. L'insecure design non è quindi un singolo errore che si può commettere come alcune vulnerabilità. bensì un'idea sbagliata fin dal concepimento. L'insecure implementation invece rappresenta una errata realizzazione del codice di un progetto che sulla carta è sicuro. L'insecure design è più pericoloso dell'insecure implementation, perché se per quest'ultimo potrebbe essere sufficiente una patch correttiva, per l'insecure design in certi casi potrebbe dover essere necessario ricreare da zero intere parti del progetto. Un esempio di insecure design è la mancata integrazione di meccanismi di sicurezza, come l'hashing o il salting delle password nella progettazione di un sistema di autenticazione. La mancanza di questi sistemi non è un errore di implementazione, ma un errore di progettazione, perché il sistema non è stato concepito per essere sicuro fin dall'inizio.

il secure product design avviene mediante due processi: product inception e product design. Il product inception è la fase iniziale e consiste nel definire i requisiti di sicurezza fin dalla nascita dell'idea del progetto, stabilendo fin da subito i controlli necessari per proteggere il sistema. il product design invece è un processo iterativo e continuo che evolve durante tutto il ciclo di vita del prodotto.

2. SECURITY PRINCIPLE

Per prevenire l'insecure design devono essere seguiti i Principi di sicurezza:

2.1 least privileges

Il principio del Least privileges (privilegio minimo) consiste nell'assegnare ad ogni utente solamente i permessi di accesso minimi per svolgere i propri compiti. In questo modo in caso di compromissione, volontaria o involontaria, i danni saranno contenuti in un'area più ristretta. Attraverso l'applicazione di tale principio si riduce la superficie di attacco e si limita la propagazione dei malware. Uno dei vantaggi più importanti è che viene reso più complesso per l'attaccante il movimento laterale nella rete.

2.2 Separation of duty

separation of duty è una politica che evita di fornire tutto il potere ad una sola persona, permettendo solo a persone autorizzate di compiere azioni sensibili, in questo modo si riduce il rischio di frodi ed errori. Una corretta separazione dei poteri permette inoltre di definire correttamente i ruoli nell'azienda e di conseguenza di poter applicare correttamente il principio del least privileges.

2.3 Defense in depth

Il principio del defense in depth consiste nel creare diversi livelli di sicurezza, in modo tale che se uno di tali livelli fallisse, l'attaccante non riuscirebbe a prendere il controllo dell'intero software ma solo di una piccola parte. inoltre l'applicazione di più strati di sicurezza permette di rispondere tempestivamente ad eventuali attacchi, se uno strato venisse attaccato verrebbero sollevati degli alert permettendo una risposta immediata. Ad esempio il primo strato di sicurezza è il firewall, se esso venisse violato ci potrebbe essere un altro strato di sicurezza, come ad esempio una segmentazione della rete, il server potrebbe trovarsi su una rete isolata rispetto ad altre componenti come il database.

2.4 Zero trust

Lo zero trust è un modello di visione generale in cui non bisogna fidarsi di nessuno. l'idea alla base è che chiunque potrebbe essere un malintenzionato e sono perciò necessari controlli di autenticazioni continue per evitare possibili attacchi. Secondo l'analista John Kindervag, bisogna passare dalla visione "Fidarsi, ma verificare sempre" a "verificare sempre, senza fidarsi". Lo zero trust si basa anche sul principio del least privileges. La mancanza di fiducia non è riservata ai soli utenti che usufruiscono del sistema, ma anche agli stessi dipendenti dell'azienda. Bisogna monitorare costantemente il sistema per individuare comportamenti anomali da parte di utenti e dipendenti dell'azienda. Se ad esempio una persona ha sempre svolto un certo compito da dentro l'azienda e all'improvviso inizia a fare tutt'altro da remoto, allora sarà necessario effettuare nuovamente un autenticazione per confermare l'identità del dipendente.

2.5 Security in the open

L'idea alla base di tale principio è che la sicurezza del software debba essere sviluppata in maniera trasparente, soprattutto nei progetti open-source. Ciò potrebbe permettere a più persone di guardare il codice ed scoprire vulnerabilità. li sviluppatori potrebbero a questo punto essere avvertiti privatamente e patchare la vulnerabilità

3 FOCUS SULLE AREE DI SICUREZZA

Il software non è composto solamente dal codice, ovviamente esso è una parte fondamentale ma esistono anche altre aree di grande importanza.

3.1 Context

bisogna valutare il contesto in cui si inseriscono le misure di sicurezza. bisogna valutare quali dati sono più importanti per l'azienda ed analizzare il profilo di rischio. il livello rischio si basa sulla **CIA** (confidenziality, Integrity, Avaliability), i tre principi che rappresentano la triade della sicurezza informatica. bisogna valutare le possibili minacce attraverso il threat modelling, che avviene per mezzo di storie relative alla sicurezza aggiunte sia durante il product inception e sia ad ogni iterazione del **product delivery**. Bisogna inoltre effettuare un business impact assesment, per assegnare un corretto livello di sicurezza ai dati aziendali. Fornire una sicurezza molto elevata per dati di poca importanza farebbero aumentare eccessivamente i costi e la complessità. al contrario una sicurezza troppo bassa potrebbe provocare danni enormi, sia ovviamente in termini economici ma anche di reputazione 2.

3.2 Components

un software non è composto solamente da codice, ma anche da librerie, framework e servizi esterni. Librerie e framework vengono decisi ad ogni ciclo di product design, sono esempi di framework React, springboot etc.. I servizi esterni sono invece scelti durante l'ideazione del progetto, ovvero durante la fase di product inception. è fondamentale scegliere bene le componenti del proprio software, bisogna infatti considerare che tali componenti potrebbero essere soggette a vulnerabilità sfruttabili da un attaccante. Per ridurre tale rischio si utilizza, oltre al threat modelling, solamente de**secure design pattern**. Bisogna inoltre considerare anche gli aspetti commerciali di tali componenti, ovvero eventuali **licenze** e **manutenzioni**. Un componente non aggiornato potrebbe infatti avere vulnerabilità note.

3.3 Connection

una volta scelte le componenti del software bisogna stabilire delle connessioni con esse. bisogna decidere come connettere il nostro software ad esempio ad un database. Tali connessioni vengono aggiunte o rimosse soprattutto durante la fase di product inception, ovvero quando ancora si stanno definendo i confini

del sistema. La modifica delle connessioni può avvenire anche durante fasi di progettazione successive a seguito di diversi motivi, che possono essere ad esempio per refactoring o per la scoperta di vulnerabilità.

3.4 Code

sono diversi i principi per la scrittura del codice

1. **Input validation:** bisogna fare in modo che vengano inseriti solamente input con tipo, formato e lunghezza validi. Ciò permette di evitare attacchi come, ad esempio, SQL injection o buffer overflow.
2. **Error handling:** implementare una corretta gestione degli errori, senza inviare informazioni sensibili ad un eventuale attaccante
3. **Authentication and authorization**: utilizzare meccanismi di autenticazione ed autorizzazione per permettere solo a determinati utenti di accedere ad informazioni sensibili
4. **Cryptography**: implementare funzioni crittografiche per proteggere i dati in transito e a riposo.
5. **Least privileges**: bisogna utilizzare il principio del privilegio minimo sul codice, in modo tale che ad esso ed alla macchina su cui viene eseguito vengano concessi i diritti di accesso minimo per svolgere le loro funzioni.
6. **Security memory management**: implementare una corretta gestione della memoria per prevenire attacchi come ad esempio buffer overflow.
7. **Avoiding hardcoded secrets**: evitare di inserire in chiaro informazioni sensibili all'interno del codice, ad esempio username e password. Conviene piuttosto utilizzare archivi sicuri per memorizzare tali informazioni, ad esempio vault esterni o fili criptati accessibili solamente solo dall'applicazione.
8. **Security testing**: effettuare test di sicurezza per individuare vulnerabilità durante lo sviluppo.
9. **Reviewing of the code**: revisionare periodicamente il codice per individuare potenziali vulnerabilità,
10. **Update**: aggiornare il codice con nuove patch di sicurezza.

3.5 Configuration

per rendere un'applicazione sicura bisogna effettuare una configurazione corretta.

1. Applicare correttamente il least privileges.

2. Implementare correttamente la difesa in profondità
3. Implementare meccanismi di emergenza in caso di errore, in questo modo si evita di esporre vulnerabilità all'esterno.
4. Avere un piano per reagire a problemi di sicurezza.
5. Aggiornare costantemente le componenti
6. Protezione dei dati sensibili sia durante il trasferimento sia a riposo
7. Utilizzare sempre connessione sicure.

4 THREAT MODELLING

il threat modelling è un'operazione fondamentale per prevenire l'insecure design. Esso non è altro che un processo in cui si cercano potenziali minacce per l'applicazione. È un processo strutturato e ripetibile eseguito principalmente durante le prime fasi di sviluppo dell'applicazione. il threat modelling non deve essere vista come un'aggiunta di sicurezza all'applicazione, bensì deve essere integrata ad essa fin dalla sua progettazione.

4.1 Threat modelling manifesto

Il threat modelling manifesto è un documento pubblicato nel 2020 e rappresenta una metodologia utilizzata per individuare, analizzare e affrontare eventuali minacce. Secondo tale manifesto, tutti coloro che abbiano a cuore la privacy dovrebbero eseguire un processo di threat modelling. Sempre secondo tale documento, per effettuare una corretta modellazione delle minacce è necessario porsi quattro domande fondamentali:

1. A cosa stiamo lavorando?
2. Cosa può andare storto?
3. Cosa faremo al riguardo?
4. Abbiamo fatto un lavoro abbastanza buono?

   Solo focalizzandoci su tali domande è possibile eseguire una corretta modellazione delle minacce. Non esiste uno standard universale per la

modellazione delle minacce. Ci sono però fasi comuni tra tutti i processi di threat modelling: **modellazione del sistema, identificazione delle minacce e risposta al rischio**.

4.2 Fasi del threat modelling

4.2.1 Modellazione del sistema

la fase di modellazione del sistema risponde alla domanda "a cosa stiamo lavorando?". Comprendere a fondo il sistema è fondamentale, perché senza una visione chiara della sua struttura e delle sue funzionalità non è possibile identificare in modo efficace le potenziali minacce. Per modellare il sistema esistono due approcci principali: data flow diagrams DFDs e il brainstorming. I DFDs consentono di rappresentare graficamente il flusso di dati attraverso il sistema, facilitando l'individuazione delle possibili vulnerabilità. Il brainstorming è un'alternativa altrettanto valida. È preferibile in quei casi in cui il team non è formato da sole persone esperte. Il brainstorming permette a tutti i membri del team di proporre nuove idee, senza i vari tecnicismi dei DFDs. Inoltre il brainstorming permette una maggiore comunicazione tra i membri del team, permettendo anche la comprensione di un sistema terminologico comune tra i membri.

4.2.2 Individuazione delle minacce

Una volta che abbiamo modellato il sistema si passa allo step successivo, ovvero l'individuazione delle minacce. Come detto in precedenza tale attività non deve essere un'aggiunta all'applicazione, ma ne deve essere le fondamenta. Tale operazione permette di rispondere alla domanda "cosa può andare storto?". Esistono varie tecniche di individuazione delle minacce ognuna con i suoi pregi e difetti, una di quelle più usate è STRIDE. STRIDE semplifica la modellazione delle minacce collegando ogni tipo di minaccia a un principio base di sicurezza, così che sia facile da capire sia per chi è tecnico sia per chi non lo è:

- **SPOOFING**
- **TAMPERING**
- **REPUDIATION**
- **INTEGRITY**
- **DENIAL OF SERVICE**
- **ELEVATION OF PRIVILEGES**

Una volta individuata una minaccia si calcola il rischio per l'azienda. Tale valore di rischio si basa sulla probabilità che tale attacco possa avvenire e sull'impatto che può comportare. Un esempio di tale calcolo di rischio è il CVSS score, una metrica che indica l'impatto di una vulnerabilità attraverso un valore numerico da 0 a 10. L'utilizzo del CVSS permette alle aziende di capire dove concentrare la maggior parte delle risorse.

4.2.3 Risposta e mitigazioni

Una volta individuate le possibili minacce dobbiamo decidere come intervenire.
In base alle risorse disponibili e all'importanza delle componenti a rischio verranno prese decisioni differenti. Le operazioni che possono essere eseguite sono le seguenti:

- **Mitigare**: si provvede ad adottare misure di sicurezza atte a diminuire la probabilità di avvenimento della minaccia.
- **Eliminare**: si rimuove completamente la componente a rischio.
- **trasferire**: Si trasferisce la responsabilità ad un'altra entità
- **accettare**: Si lascia inalterata la componente, lasciando quindi la vulnerabilità. Tale scelta viene fatta quando in base ai requisiti aziendali non è possibile attuare le altre operazioni.

5 ECOMMERCE – DEMO

5.1 INTRODUZIONE ALLA DEMO

È stata realizzata una demo per mostrare concretamente alcune vulnerabilità relative all'OWASP top 10 A04-2021 Insecure design.

La demo consiste in un semplice e-commerce di elettronica varia. All'interno di tale e-commerce sarà obbligatorio autenticarsi e successivamente si avrà la possibilità di:

- Visualizzare i prodotti disponibili
- Effettuare ordini
- Visualizzare gli ordini effettuati
- Depositare soldi per ricaricare il proprio conto.

L'applicazione è stata progettata con due specifiche vulnerabilità di logica che possono portare a pesanti conseguenze sia per l'azienda che per i suoi utenti.
Le vulnerabilità rientrano nelle seguenti categorie, associate a un design insicuro:

- CWE-602 - Client-Side Enforcement of Server-Side Security
- Insecure Direct Object Reference (IDOR)

5.2 IDOR

5.2.1 Definizione IDOR

L'IDOR (Insecure Direct Object Reference) è una vulnerabilità che si verifica quando un'applicazione web espone pubblicamente un riferimento diretto ad un determinato oggetto interno di un utente senza un'adeguata verifica dei permessi di accesso a tale oggetto.

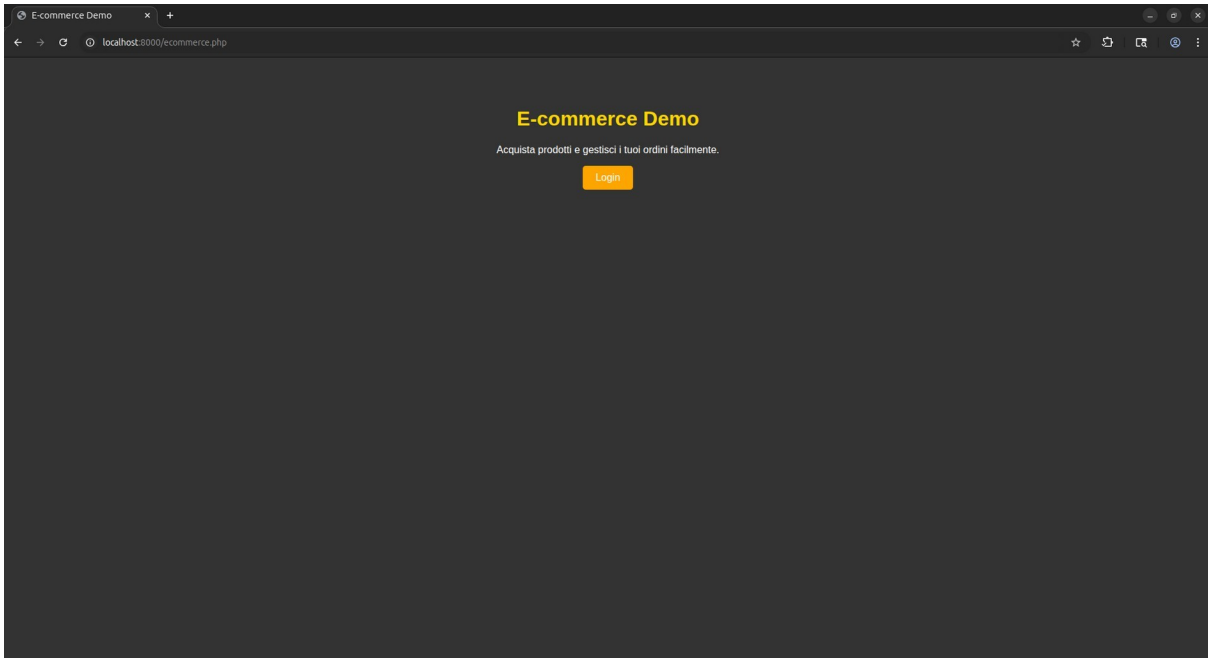
L'applicazione web potrebbe ad esempio inserire un riferimento all'oggetto all'interno dell'URL. Se l'applicazione non effettua controlli di autenticazione per l'utente proprietario dell'oggetto, allora chiunque potrebbe modificare l'URL ed accedere alla risorsa.

L'IDOR è un chiaro esempio di Insecure Design. Non si tratta infatti di un errore di implementazione, bensì di un errore logico della sicurezza. Si tratta di

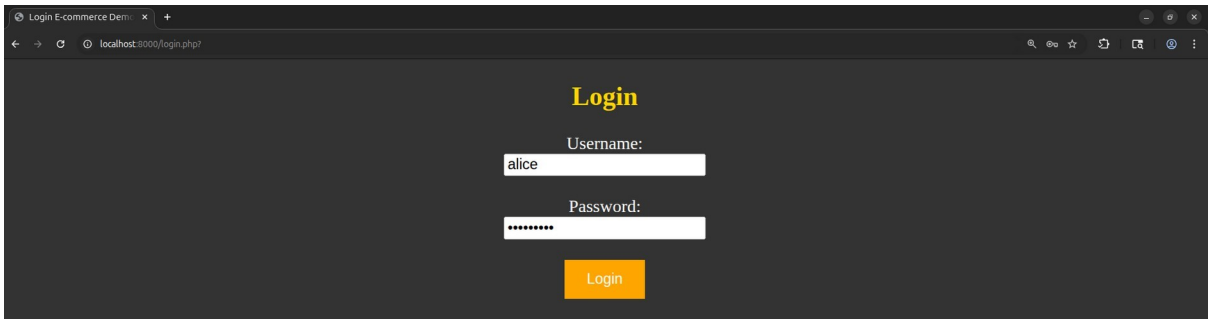
eccesso di fiducia verso gli utenti che usufruiscono del sistema. L'applicazione web da per scontato che l'utente che tenta di accedere ad un oggetto me è anche il proprietario.

5.2.2 IDOR Attacco

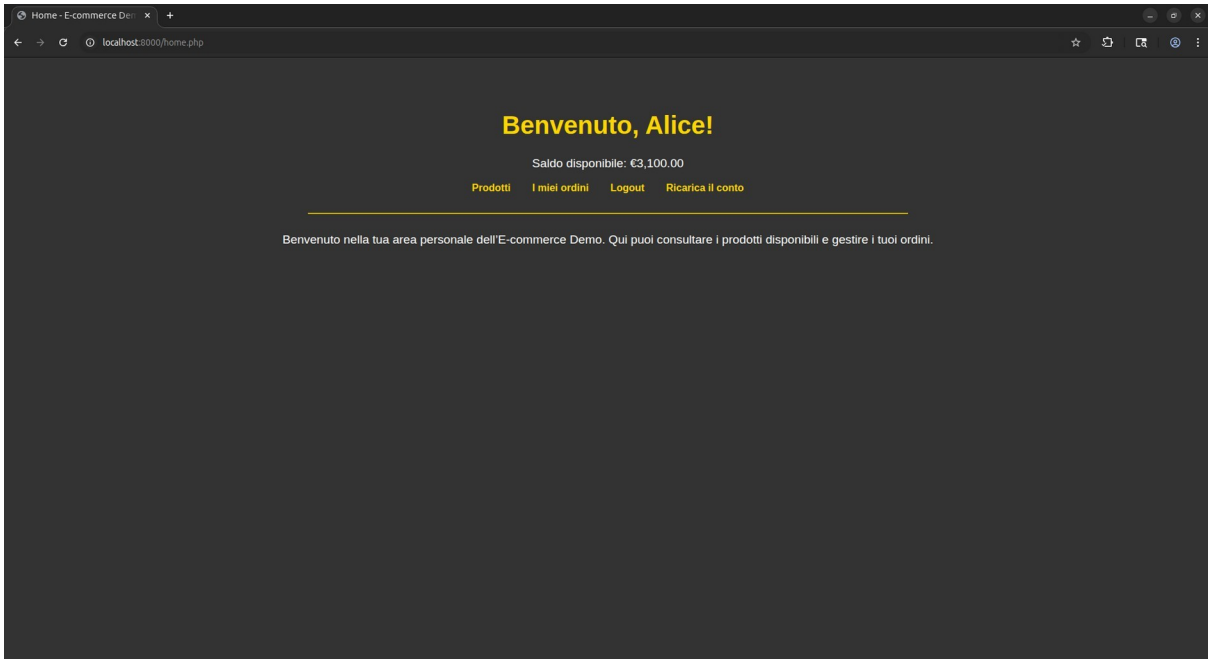
All'interno del progetto è stata sfruttata tale vulnerabilità per mostrare gli ordini effettuati da altri utenti.

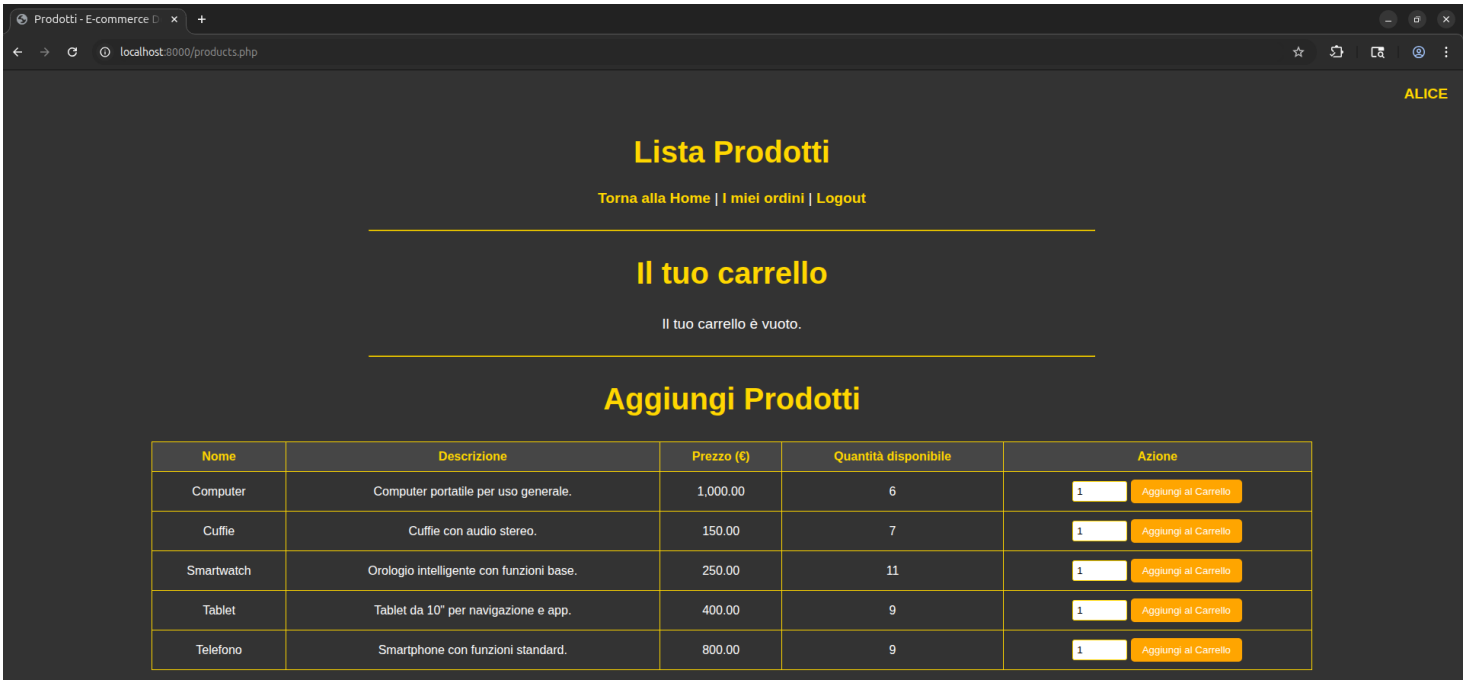


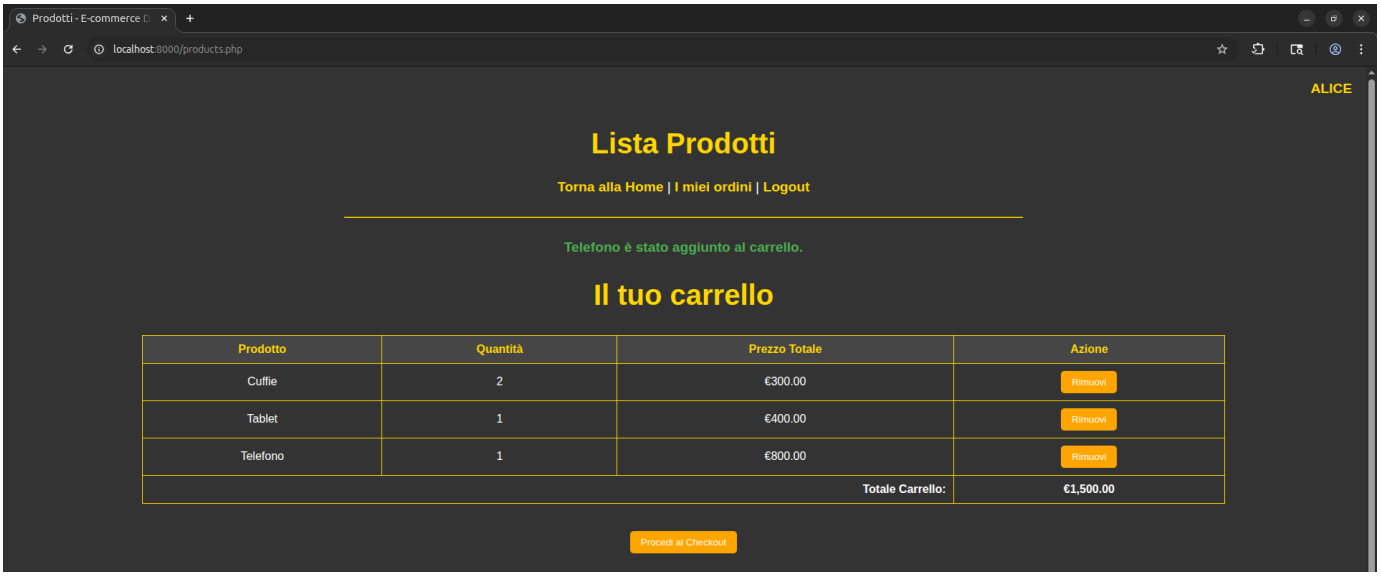
È stato preso un utente registrato nel database e ne è stato effettuato il login utilizzando le credenziali dell'utente Alice (username: Alice, password: alicepass).

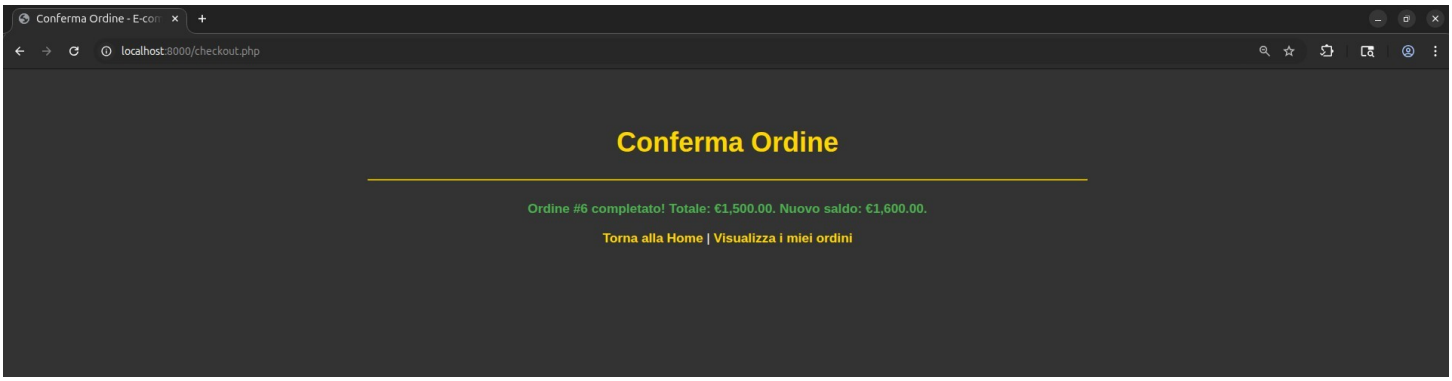


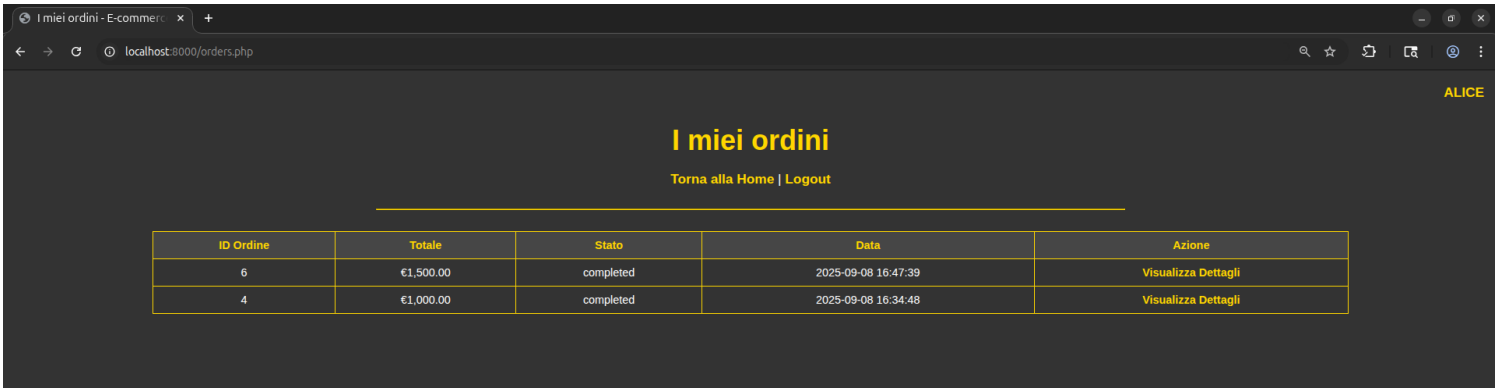
Una volta effettuato il login si è avuto accesso alla home del utente. Da qui vi è la possibilità di effettuare diverse operazioni. Per dimostrare la presenza della vulnerabilità IDOR è necessario effettuare un ordine e verificare se un altro utente riesce ad accedergli.


Home - E-commerce Dem
localhost:8000/home.php
Benvenuto, Alice!
Saldo disponibile: €3,100.00
Prodotti
I miei ordini
Logout
Ricarica il conto
Benvenuto nella tua area personale dell'E-commerce Demo. Qui puoi consultare i prodotti disponibili e gestire i tuoi ordini.


Prodotti - E-commerce D
localhost:8000/products.php
ALICE
Lista Prodotti
Torna alla Home | I miei ordini | Logout
Il tuo carrello
Il tuo carrello è vuoto.
Aggiungi Prodotti
Nome
Descrizione
Prezzo (€)
Quantità disponibile
Azione
Computer
Computer portatile per uso generale.
1,000.00
6
Cuffie
Cuffie con audio stereo.
150.00
7
Smartwatch
Orologio intelligente con funzioni base.
250.00
11
Tablet
Tablet da 10" per navigazione e app.
400.00
9
Telefono
Smartphone con funzioni standard.
800.00
9
Aggiungi al Carrello

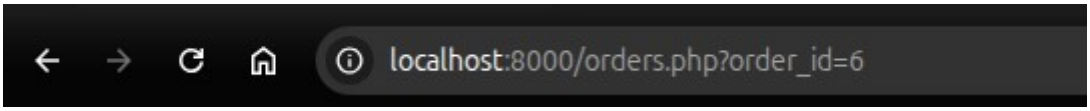






Una volta che l'ordine è stato effettuato e salvato nel database è possibile accedere ai dettagli di tale ordine, ed è qui che viene mostrata la vulnerabilità

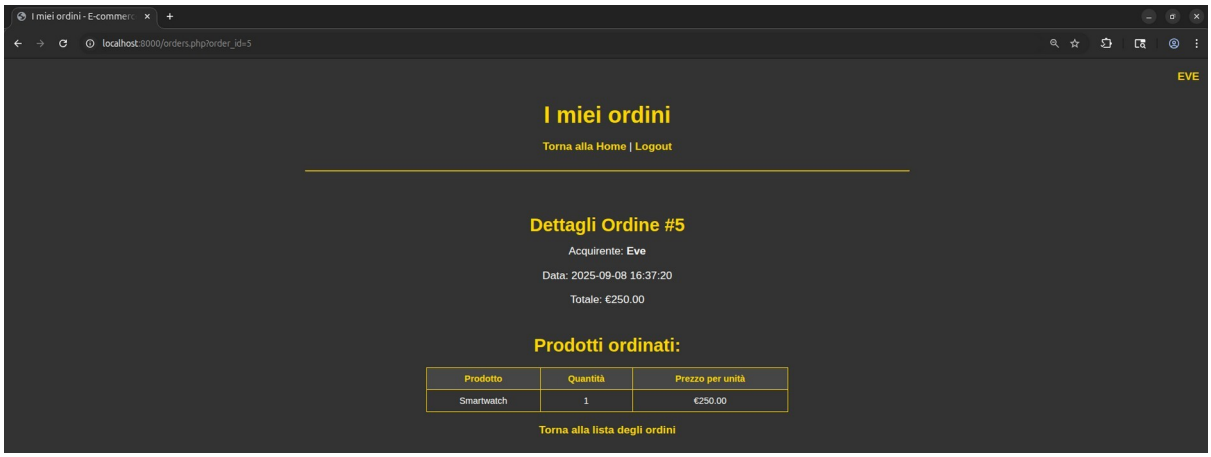




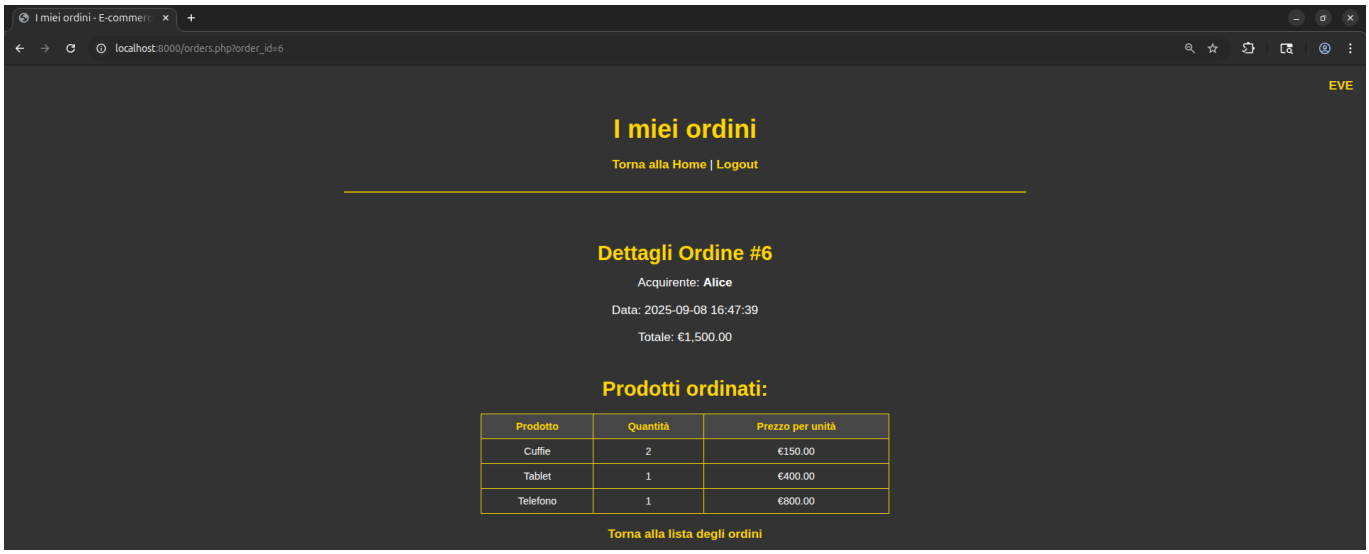
All'interno dell'URL è presente l'id dell'ordine dell'utente, per un utente normale ciò potrebbe sembrare insignificante, ma per qualcun altro potrebbe essere il segnale per un possibile attacco. è stato quindi effettuato l'accesso con un altro utente per sfruttare tale vulnerabilità.

L'utente malintenzionato potrebbe inizialmente non sapere quale sia la sintassi dell'URL per poter accedere ad un oggetto, poiché essa viene mostrata solo dopo aver effettuato un acquisto.

L'utente Eve potrebbe quindi effettuare un acquisto ed accedere ai dettagli del proprio ordine.



Eve una volta notato l'URL potenzialmente vulnerabile, decide di modificare l'id dell'ordine, modificando quindi l'url http://localhost:8000/orders.php?order_id=6 e inserendo l'id dell'ordine di Alice. Riuscendo così ad accedere all'ordine effettuato in precedenza da Alice.



5.2.3 Conseguenze

Lo sfruttamento dell'IDOR potrebbe portare ad impatti più o meno grandi. Nel caso della seguente demo vi è un attacco alla privacy degli utenti, che anche se possa sembrare un problema di poco conto, in realtà è un attacco che danneggia pesantemente la reputazione di un'azienda. In un sito web con funzionalità più complesse, un attaccante potrebbe:

- Manipolare o cancellare un ordine
- Cambiare la destinazione della spedizione verso il proprio domicilio

Ciò porta ad una violazione del principio del Least privileges, in cui ogni utente deve poter avere l'accesso solamente alle risorse di cui ha strettamente bisogno.

5.2.4 Mitigazioni

Per mitigare la vulnerabilità IDOR è necessario attuare delle misure di sicurezza aggiuntive. Nel codice la vulnerabilità nasce all'interno di orders.php è dovuta al fatto che la query prende in input solamente l'ID dell'ordine, dando per scontato che tale ordine appartenga all'utente corretto. Per l'applicazione è sufficiente sapere che l'utente sia autenticato.

```
$stmt = $conn->prepare(query: "SELECT o.id, o.total, o.created_at, u.nome
                        FROM orders o
                        JOIN users u ON o.user_id = u.id
                        WHERE o.id = ?");
$stmt->bind_param(types: "i", var: &$order_id);
```

Modifichiamo quindi la query in modo tale che se l'ID dell'ordine e l'ID dell'utente non corrispondono nella stessa riga del database, la query non restituirà alcun risultato.

```
$stmt = $conn->prepare(query: "SELECT o.id, o.total, o.created_at, u.nome
                        FROM orders o
                        JOIN users u ON o.user_id = u.id
                        WHERE o.id = ? AND o.user_id = ?");
$stmt->bind_param(types: "ii", var: &$order_id,vars: &$user_id);
```

5.3 CWE-602 - Client-Side Enforcement of Server-Side Security

5.3.1 Definizione CWE-602

La CWE-602 Client-side Enforcement of Server-Side Security è una vulnerabilità di design che si verifica quando i controlli di sicurezza vengono effettuati lato client ed il server non effettua nuovamente controlli dal suo lato. Il problema è che il browser dell'utente non può essere considerato sicuro, poiché è facilmente manipolabile tramite strumenti e proxy intercettanti come Burpsuite. Utenti normali quasi sicuramente non si accorgeranno mai di tale debolezza, ma utenti malintenzionati potrebbero invece accorgersene per bypassare vari controlli di sicurezza.

L'attaccante può infatti bypassare i controlli di validazione dei dati lato client, fornendo al server dei dati corrotti e potenzialmente inattesi da esso, causando un vantaggio personale o un malfunzionamento del sistema.

5.3.2 CWE-602 Attacco

All'interno del progetto è stata sfruttata questa vulnerabilità per ottenere un vantaggio economico all'attaccante sfruttando un errore nella logica di gestione degli ordini.

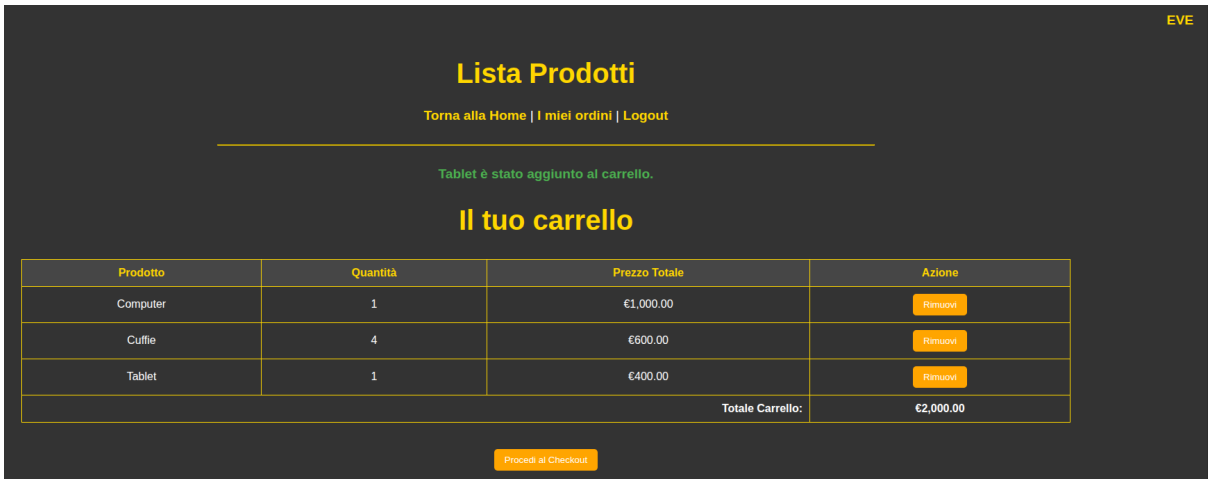
Il difetto di design risiede nel fatto che l'applicazione non esegue alcun controllo lato server per verificare la correttezza del prezzo di un ordine. Il prezzo viene deciso solamente lato client, il server si occuperà solamente di

confermare e memorizzare l'ordine. Il prezzo viene inviato dal client tramite un campo nascosto all'interno di products.php e dato per corretto una volta inviato il form al checkout.php.

```
<form method="post" action="checkout.php">
    <input type="hidden" name="total_price" value="<?php echo $cart_total; ?>">
    <input type="submit" value="Procedi al Checkout">
</form>
```

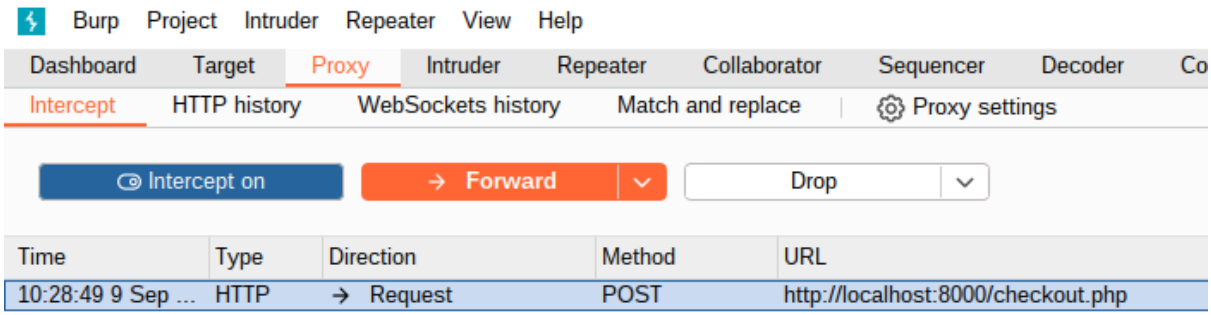
Un attaccante potrebbe sfruttare questa vulnerabilità piazzando un ordine normalmente, riempiendo il carrello e preparandosi a confermare l'acquisto.





L'ordine di Eve sarebbe dovuto essere di €2000, ma sfruttando questa debolezza abbasserà il costo totale a €1.

Per realizzare l'attacco è stato utilizzato il software **burpsuite,** un proxy intercettante che funge da mediatore tra il browser ed il server web. Attraverso tale software viene intercettata la richiesta http del pagamento prima che arrivi al server web. Si procede quindi preparando burpsuite ad intercettare la richiesta e si clicca su checkout.



Request

Pretty Raw Hex

```
10 Content-Type: application/x-www-form-urlencoded
11 Upgrade-Insecure-Requests: 1
12 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/139.0.0.0 Safari/537.36
13 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,applicatio
14 Sec-Fetch-Site: same-origin
15 Sec-Fetch-Mode: navigate
16 Sec-Fetch-User: ?1
17 Sec-Fetch-Dest: document
18 Referer: http://localhost:8000/products.php
19 Accept-Encoding: gzip, deflate, br
20 Cookie: PHPSESSID=30f3337b4f918035029fefccc22c7314
21 Connection: keep-alive
22
23 total_price=2000
```

All'attaccante basta modificare il valore del prezzo nella richiesta d'ordine per completare l'acquisto al prezzo desiderato.

Request

Pretty Raw Hex

```
10 Content-Type: application/x-www-form-urlencoded
11 Upgrade-Insecure-Requests: 1
12 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/139.0.0.0 Safari/537.36
13 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,applicatio
14 Sec-Fetch-Site: same-origin
15 Sec-Fetch-Mode: navigate
16 Sec-Fetch-User: ?1
17 Sec-Fetch-Dest: document
18 Referer: http://localhost:8000/products.php
19 Accept-Encoding: gzip, deflate, br
20 Cookie: PHPSESSID=30f3337b4f918035029fefccc22c7314
21 Connection: keep-alive
22
23 total_price=1
```

L’attacco è ormai completato, basta solamente inviare la richiesta.





5.3.3 Conseguenze

Le conseguenze relative a tale sfruttamento della debolezza possono essere molto gravi.

Si ha ovviamente una perdita finanziaria dovuto al pagamento irrisorio, ciò può portare enormi danni per un azienda. Inoltre la compromissione del prezzo provoca anche un alterazione della registrazione degli ordini all'interno del database.

5.3.4 Mitigazioni

Per mitigare il problema bisogna innanzitutto comprendere dove nasce.

Il problema principale è che il sistema riceve un valore total_price inviato da un campo nascosto all'interno del form di procedimento al checkout di products.php. Il server una volta ricevuto il valore di total_price, si fida ciecamente di esso e rimuove dal saldo dell'utente solamente il prezzo ricevuto.

checkout.php products.php

```
185             <tr>
186                 <td colspan="3" style="text-align:right;"><b>Totale Carrello:</b></td>
187                 <td><b>€<?php echo number_format(num: $cart_total, decimals: 2); ?></b></td>
188             </tr>
189         </table>
190         <br>
191         <form method="post" action="checkout.php">
192             <input type="hidden" name="total_price" value="<?php echo $cart_total; ?>">
193             <input type="submit" value="Procedi al Checkout">
194         </form>
```

checkout.php

php_shop > checkout.php

```
1   <?php
2   session_start();
3   require_once 'config.php';
4
5   if (!isset($_SESSION['user_id'])) {
6       header(header: "Location: login.php");
7       exit;
8   }
9
10  $user_id = $_SESSION['user_id'];
11  $message = '';
12  $error = '';
13
14  if ($_SERVER['REQUEST_METHOD'] === 'POST') {
15
16      $total_price_from_client = $_POST['total_price'];
17
```

La soluzione a tale problema è quella di far effettuare al server il controllo del prezzo totale dell'ordine basandosi sui prezzi presenti nel database.

Il codice di products.php rimane inalterato, ciò che cambia è il codice di checkout.php.

Il server non si fida più del prezzo fornito dal client, ma ricalcola il prezzo durante il checkout. Viene effettuata una query per ogni elemento all'interno del carrello in cui si prende il prezzo e la quantità disponibile in magazzino del prodotto.

```php
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    if (empty($_SESSION['cart'])) {
        $error = "Il tuo carrello è vuoto. Impossibile completare l'ordine.";
    } else {
        $cart_total = 0;
        foreach ($_SESSION['cart'] as $product_id => $item) {
            $stmt = $conn->prepare(query: "SELECT price, quantity FROM products WHERE id = ?");
            $stmt->bind_param(types: "i", var: &$product_id);
            $stmt->execute();
            $result = $stmt->get_result();
            if ($row = $result->fetch_assoc()) {
                if ($row['quantity'] < $item['quantity']) {
                    $error = "Quantità insufficiente in magazzino per il prodotto: " . htmlspecialchars(string: $item['name']);
                    break;
                }
                $item_price = $row['price'];
                $cart_total += $item_price * $item['quantity'];
            } else {
                $error = "Uno o più prodotti nel carrello non sono validi.";
                break;
            }
        }
        $stmt->close();
```

6 CONCLUSIONI

Attraverso questo progetto si è mostrato come una logica di design debole possa portare a conseguenze critiche. Le vulnerabilità mostrate non sono errori isolati di implementazione, bensì frutto di uno studio non corretto del design.

Se prendiamo infatti il codice con un design non sicuro scritto prima della mitigazione non sono presenti errori di implementazione, essa è assolutamente corretta ma per una logica sbagliata.

La sicurezza non deve essere una toppa messa a lavoro finito, ma un requisito fondamentale durante tutto il ciclo di sviluppo del sistema.

BIBLIOGRAFIA

https://owasp.org/Top10/it/A04_2021-Insecure_Design/
https://cheatsheetseries.owasp.org/cheatsheets/Secure_Product_Design_Cheat_Sheet.html
https://cheatsheetseries.owasp.org/cheatsheets/Threat_Modeling_Cheat_Sheet.html https://www.threatmodelingmanifesto.org/
https://www.securitycompass.com/blog/stride-in-threat-modeling
https://shostack.org/resources/threat-modeling
https://www.akamai.com/it/glossary/what-is-zero-trust#:~:text=Il%20modello%20Zero%20Trust%20%C3%A8,solo%20se%20ritenuto%20espressamente%20necessario.

https://cheatsheetseries.owasp.org/cheatsheets/Insecure_Direct_Object_Reference_Prevention_Cheat_Sheet.html